

Double Degree Program

On Using Wave Function Collapse for Depicting Software Systems

Simone Lesinigo

USI Advisor:	Prof. Dr. Michele Lanza
USI Co-advisor:	Dr. Marco Raglianti
UniMiB Co-advisor:	Prof. Dr. Claudia Raibulet

Academic Year 2025–2026

1. Introduction and Objectives

Program comprehension is as old as software engineering itself. As software systems grow and become more complex, developers spend much of their time reading code, exploring its structure, and understanding how different parts work together. Studies suggest that program comprehension takes up around 70% of a developer's time, creating a significant cognitive burden.

Software visualization can help developers understand complex systems by showing their structure and relationships. However, creating useful visualizations for large software systems is still challenging. Two main problems need to be addressed: the *Layout Problem*, which concerns the placement of software entities, and the *Connectivity Problem*, which concerns how to draw relationships while avoiding excessive edge crossings and visual clutter.

The goal of this thesis is to investigate how Wave Function Collapse (WFC) can be adapted to generate software visualizations. WFC is a procedural generation algorithm originally inspired by quantum mechanics and widely used in game development. It works on a grid where each cell can contain different tiles. Local rules define which tiles can be placed next to each other. By repeatedly selecting a cell and choosing one of its possible tiles, WFC gradually builds a complete grid.

In this thesis, WFC is adapted to generate paths between software entities. Each tile represents a part of a path or an empty space, and local rules ensure that neighboring tiles are compatible. However, these rules alone cannot guarantee that paths reach their intended destinations or that the overall visualization preserves all software relationships. To address this problem, additional mechanisms are introduced to guide path generation and satisfy these global constraints while discouraging edge crossings and visual clutter.

2. From Code to Visualization

To achieve this goal, we designed a new approach and implemented it in a tool called Wassis, developed in Pharo, a Smalltalk environment. The process starts from the source code and progressively builds the final visualization through four main phases, shown in Figure 1.

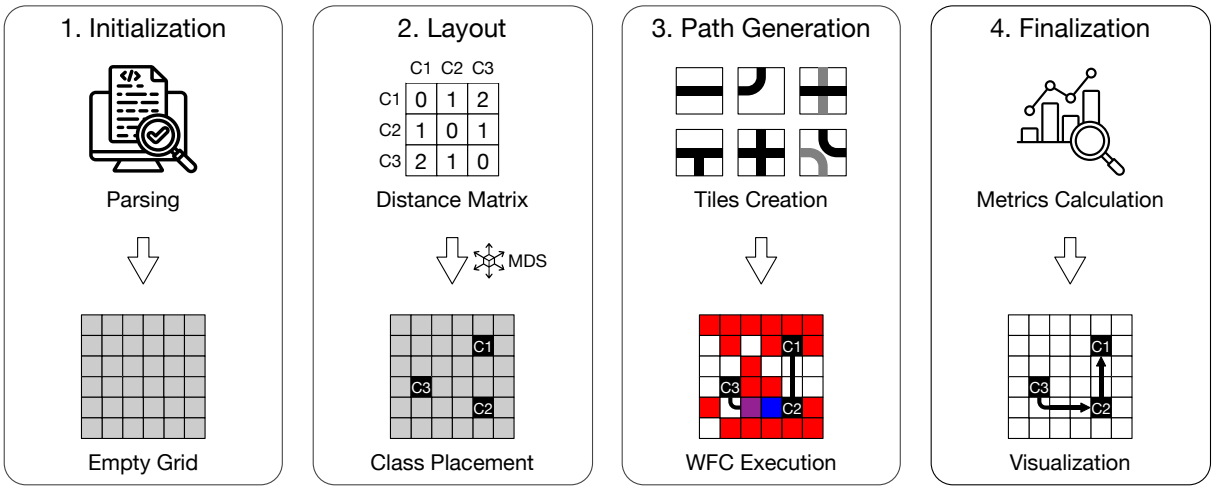


Figure 1: Overview of the Wassis pipeline. The four main phases are source code parsing, layout generation using Multidimensional Scaling (MDS), path generation using Wave Function Collapse (WFC), and final metric calculation and visualization rendering.

1. Initialization: The process starts by parsing the source code to extract classes and their relationships. A two-dimensional grid is then created. Each cell initially contains all possible tiles, including empty tiles and different path segments.

2. Layout using Multidimensional Scaling (MDS): To solve the *Layout Problem*, a distance matrix is built from the relationships between classes, with smaller distances representing stronger connections. The Floyd-Warshall algorithm is then used to enrich the distance matrix by considering indirect connections between classes.

Multidimensional Scaling (MDS) then converts these distances into two-dimensional coordinates. Each class is represented by a square whose size depends on the number of incoming and outgoing paths.

3. Path Generation using WFC: To solve the *Connectivity Problem*, relationships are represented as paths on the grid. Relationships with the same destination and type are merged into a single path and assigned a unique ID.

Several changes were introduced to adapt WFC to software visualization:

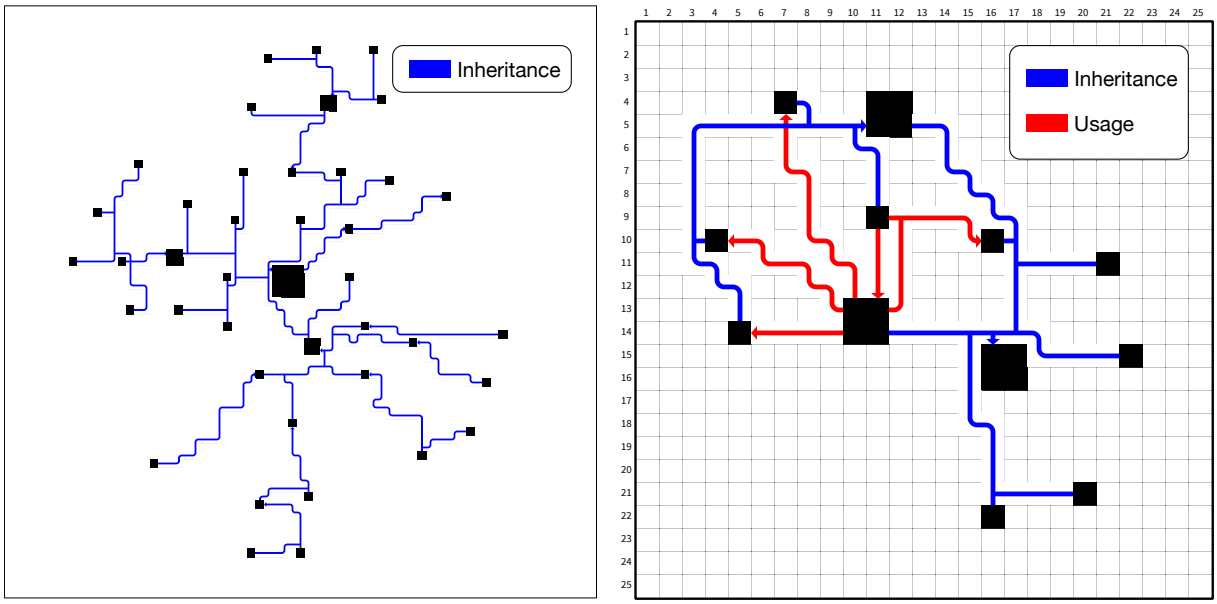
- *Dynamic Placeholders:* To avoid creating too many tiles for systems with many different path IDs, we introduced a placeholder system. It assigns path identities dynamically as cells are collapsed into path tiles.
- *Max-Entropy Heuristic:* Standard WFC implementations often select the cell with the lowest entropy. In our case, this would cause empty cells to be collapsed first, wasting space that could instead be used by paths. Wassis therefore selects the cell with the highest entropy, giving priority to path generation.

- *Directional Exits and Dynamic Tile Weights:* Entry and exit directions are fixed on class borders to define where paths start and end. Their directions are chosen to facilitate path entry and exit and help paths connect to their intended endpoints. Tile weights are also adjusted dynamically to favor tiles that move a path closer to its destination.
- *Backtracking Mechanisms:* Local constraints can lead to contradictions when a cell has no valid tiles left. To handle this problem, we introduced different backtracking strategies. These strategies allow WFC to undo previous decisions or restart the generation process with a larger grid.

4. Finalization: Once a valid configuration is found, the system calculates metrics such as path length, edge crossings and execution time. The final result is then rendered as an interactive visualization.

3. Results Obtained

To evaluate the approach, we present two examples (Figure 2): the *Magnitude* class hierarchy in Pharo, which contains only inheritance relationships, and a subset of the *JetUML* framework, which contains both inheritance and usage relationships. These examples show that the approach can be used to generate software visualizations.



(a) *Magnitude* hierarchy. (b) *JetUML* subset.

Figure 2: Visualizations generated by Wassis.

The experiments highlighted both the advantages and key limitations of the approach:

- **Visual Clarity and Clutter Reduction:** Wassis produces compact, organic layouts that distribute entities in multiple directions around central classes, avoiding the extreme width typical of traditional top-down trees in tools like PlantUML. A major strength is *path merging*, which combines relationships of the same type and destination into a single line, drastically reducing visual clutter. Additionally, WFC's local constraints effectively discourage crossings.

- **System Size and Scalability:** The primary limitation of the tool is the size of the software system that can be visualized. As the number of entities and connections increases, the grid size expands, leading to an exponential growth in the search space. Consequently, generating valid configurations for larger or denser systems becomes computationally expensive, requiring significant execution time.
- **Impact of Configuration Parameters:** Configuration choices strongly affect WFC performance. Parameters such as *Path Boost* were essential for guiding paths toward their targets and enabling WFC to find valid solutions. Similarly, the minimum distance between classes balances the space available for paths against the overall search complexity.

Overall, while computational cost currently limits the scale of systems that can be visualized, the results confirm that WFC can be adapted to generate clean and structured visualizations.

4. Future Work

Several improvements could be explored to address the current limitations and make Wassis more efficient and suitable for larger software systems.

- **Performance and Scalability:** The reduction of possible tiles in neighboring cells after a cell is collapsed could be improved by using a GPU to process several cells in parallel. Another possibility is to divide large systems into smaller subgrids and solve each one separately. However, paths crossing the boundaries between subgrids would need to be defined before or during generation to keep the different subgrids consistent. This could reduce the size of each problem, but the division of the grid and the handling of these connections would require further study.
- **More Flexible Tile Changes:** WFC could be extended to allow changes to cells that have already been collapsed. For example, when a path is blocked, a straight tile could be changed into a crossing or a junction instead of triggering backtracking. This could reduce the number of failed attempts.
- **Automatic Parameter Configuration:** Wassis currently requires manual configuration of parameters. Machine learning or heuristics based on previous executions could help select suitable parameters automatically, depending on the size, density, and relationship types of the software system to be visualized.
- **Evaluation with Developers:** The current evaluation mainly focuses on structural metrics, such as edge crossings and path lengths. A user study involving software developers is needed to understand whether the generated visualizations actually improve program comprehension. Such a study could measure how easily developers understand and navigate software systems using Wassis.